



Métodos Funcionais de Arrays

Aula 08 · Bloco 2 — Estruturas de Dados

map, filter, reduce — o jeito profissional de tratar listas

Nesta aula

Na aula passada você **reinventou** o `map` e o `filter` nos exercícios.

Agora, os oficiais:

1. `forEach` — percorrer
2. `map` — transformar
3. `filter` — selecionar
4. `find`, `some`, `every` — perguntar
5. `reduce` — resumir
6. **Encadeamento** — o superpoder

forEach — percorrer

```
const nomes = ["Ana", "Bruno", "Carla"];

nomes.forEach((nome, indice) => console.log(`${indice + 1}. ${nome}`));
```

Como o `for...of`, em estilo funcional. **Não retorna nada** — serve só para "fazer algo com cada item".

map — transformar

Devolve um **novo array**, do **mesmo tamanho**, com o resultado do callback:

```
const precos = [100, 250, 80];
const comDesconto = precos.map((p) => p * 0.9);
console.log(comDesconto);           // [90, 225, 72]

const soNomes = turma.map((aluno) => aluno.nome);
console.log(soNomes);               // ["Ana", "Bruno"]
```

💡 Regra de bolso: entrou com 5, sai com 5. O **map** **transforma**, nunca descarta.

filter — selecionar

Devolve um novo array **só com os que passaram** no teste:

```
const notas = [8.5, 6.2, 9.1, 4.0, 7.7];

const aprovadas = notas.filter((n) => n >= 7);
console.log(aprovadas);           // [8.5, 9.1, 7.7]

const alunosAprovados = turma.filter((a) => a.media >= 7);
```

O callback do **filter** responde uma pergunta de **sim ou não**.

`find`, `some`, `every` — perguntar

```
turma.find((a) => a.nome === "Bruno");    // o OBJETO, ou undefined
turma.some((a) => a.media < 7);           // EXISTE algum abaixo de 7?
turma.every((a) => a.media >= 4);         // TODOS acima de 4?
```

Método	Devolve	Responde
<code>find</code>	o elemento	"qual é?"
<code>some</code>	<code>true</code> / <code>false</code>	"existe pelo menos um?"
<code>every</code>	<code>true</code> / <code>false</code>	"vale para todos?"

reduce — resumir tudo em um valor

```
const notas = [8, 6, 9, 7];

//          acumulador ↓   ↓ item           ↓ valor inicial
const soma = notas.reduce((acc, nota) => acc + nota, 0);
console.log(soma);           // 30
```

Passo a passo:

```
acc=0, nota=8 → 8      acc=14, nota=9 → 23
acc=8, nota=6 → 14     acc=23, nota=7 → 30
```

reduce sobre objetos

```
const carrinho = [
  { produto: "Mouse",    preco: 80,   qtd: 2 },
  { produto: "Teclado",  preco: 150,  qtd: 1 },
];

const total = carrinho.reduce((acc, item) => acc + item.preco * item.qtd, 0);
console.log(total);      // 310
```

O **reduce** é o mais poderoso — e o que mais exige prática. Comece sempre perguntando: **"o que estou acumulando, e a partir de qual valor inicial?"**

Encadeamento

Como `map` e `filter` **devolvem arrays**,
dá para emendar — e ler o código como uma frase.

```
turma
  .filter((a) => a.media >= 7)
  .map((a) => a.nome.toUpperCase());

// ["ANA", "CARLA"]
```

"Dos aprovados, pegue os nomes em maiúsculas."

for ou método funcional?

Os dois funcionam. A diferença é o que o código **comunica**:

- O `for` diz **como** — índices, contadores, incrementos;
- O método funcional diz **o quê** — filtrar, transformar, resumir.

Em geral o funcional fica mais legível, e é o padrão do mercado.

⚠ Mas se a lógica ficar torta dentro de um `reduce`, um `for` honesto é melhor que um funcional ilegível. Legibilidade ganha da moda.

Exercícios da aula

Na pasta `aula-08/`, sobre o mesmo dataset de 5 alunos:

1. `ex01.js` — `map` : `"Ana (SI)"` e `{ nome, situacao }` ;
2. `ex02.js` — `filter` : alunos de SI; aprovados com menos de 5 faltas;
3. `ex03.js` — `find` / `some` / `every` ;
4. `ex04.js` — `reduce` : média geral, total de faltas, maior média;
5. `ex05.js` — **encadeando**: `filter` → `reduce`, e `filter` → `map` → `sort`;
6. **Desafio** 🌶️ `ex06.js` — boletim inteiro com `map` + `join` , num só `console.log` .

Fim do Bloco 2

Arrays, objetos e os métodos que transformam coleções inteiras de uma vez.

Bloco 3 — O Navegador e o DOM

Na próxima aula o seu código ganha uma **tela**.